VERIQTA

Git & GitHub:

Production Engineering Playbook

Repository Architecture, CI/CD, Security, Release Management & Incident Recovery

Git | GitHub



- Core Git Workflows
- Collaborative Development
- Branching and Merging Strategies
- Code Review and Pull Requests
- CI/CD with GitHub Actions
- Release, Tagging and Rollback
- Security, Access and Governance
- Real-World Projects and Fixes



@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

The Commit That Broke Production



Understand commits, snapshots, working trees, staging, and how Git preserves changes.

The Git Workflow

Working Tree (Your files) → Staging Area (Changes to commit) → Commit (Snapshot) → Repository (All snapshots)

What You Will Learn

- What a commit really is
- How Git takes a snapshot of your code
- The roles of the working tree, staging area, and repository
- How Git preserves your change history
- How a single bad commit can break production

Hands-On Project

Recreate a production failure in a safe environment:

1. Create a sample application
2. Make a change that breaks it
3. Commit and inspect the history
4. Identify the bad commit
5. Restore the last stable version

Outcome

You will be able to use Git confidently to track changes, investigate issues, and recover from bad commits.

Your Repository Is Not Your Backup Strategy



Build a local repository, connect a remote, and recover from a simulated laptop failure.

What You Will Build

Local Repository
(Your laptop)
→
Remote Repository
(GitHub)
→
Simulate Laptop Failure
(Lost or crashed)
→
Recover Your Work
(From GitHub)

Key Concepts

- Local vs. remote repository
- Why a Git repository is not a backup
- Pushing and pulling changes
- Cloning to a new machine
- Verifying a complete recovery

Hands-On Project

1. Initialize a Git repository locally
2. Connect it to a GitHub remote
3. Commit and push your project
4. Simulate a laptop failure (remove the local repo)
5. Recover your work by cloning from GitHub

Outcome

You will understand the difference between version control and backup, and confidently recover your work from a remote.

The Secret That Entered Git History



Investigate accidentally committed credentials, containment, rotation, and history cleanup.

The Investigation Flow

Secret Committed (By mistake) → **Detect & Investigate** (Find the commit) → **Contain** (Limit exposure) → **Rotate** (Replace the credentials) → **Clean Up** (Remove from Git history)

Key Concepts

- How secrets end up in Git history
- Identifying the exact commit
- Containing the exposure quickly
- Rotating compromised credentials
- Cleaning Git history safely
- Preventing future secret leaks

Hands-On Project

1. Commit a sample secret (e.g., API key)
2. Detect the secret in history
3. Revoke and rotate the credential
4. Remove the secret from history (using git filter-repo)
5. Verify that the secret is gone

Outcome

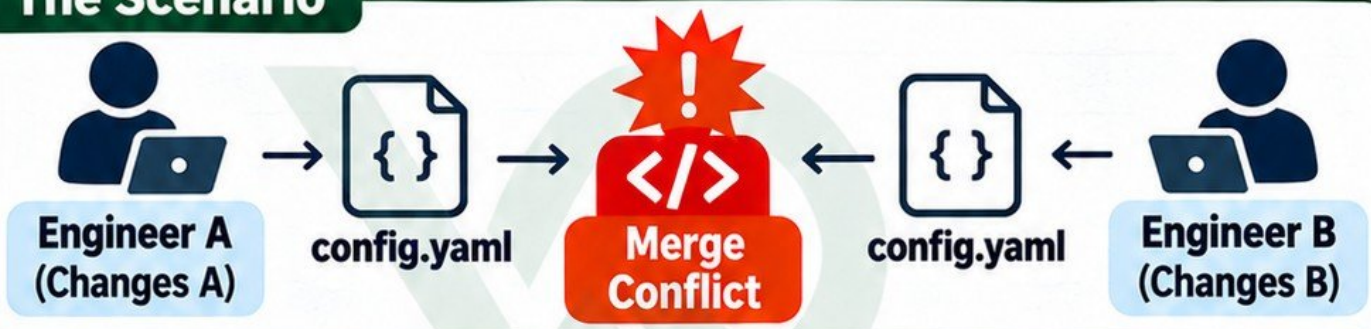
You will know how to respond to a real credential leak, secure your repository, and prevent it from happening again.

Two Engineers Changed the Same Production Configuration



Reproduce a merge conflict, investigate both changes, and resolve it without losing behavior.

The Scenario



Key Concepts

- What a merge conflict is
- Why it happens in team workflows
- Reading and understanding conflict markers (<<<<<<<, =======, >>>>>>>)
- Comparing both changes
- Resolving the conflict safely
- Testing the final configuration

Hands-On Project

1. Create a shared repo with a config file (e.g., config.yaml)
2. Make a change from two separate branches
3. Merge and reproduce the conflict
4. Investigate both changes and decide what to keep
5. Resolve, commit, and verify that both behaviors work

Outcome

You will confidently handle merge conflicts in real team scenarios without deleting or breaking existing work.

The Branch That Drifted for Three Weeks



Compare merge, rebase, squash, and short-lived branches in a realistic team workflow.

The Scenario



Compare the Strategies

Merge	Keeps full history (safe, non-destructive).
Rebase	Replays your changes (cleaner, linear history).
Squash	Combines commits into one (simpler history).
Short-Lived Branches	Keep branches small and merge early (less drift, fewer conflicts).

Hands-On Project

1. Create a repo and two feature branches
2. Let the branches drift (simulate 3 weeks of changes)
3. Try merge, rebase, and squash
4. Compare the commit history and conflict resolution
5. Choose the best approach and merge to main

Outcome

You will know when to use merge, rebase, squash, and how to use short-lived branches to avoid long, risky drift.

The Pull Request That Passed Review but Failed Deployment



Build a pull request with automated tests, review requirements, and deployment checks.

The Flow

Create Change (Feature Branch) → Open Pull Request (PR) → Code Review (Approved) → Automated Tests (Passed) → Deployment (Failed)

What You Will Learn

- The complete pull request lifecycle
- How to set up and run automated tests
- Review requirements that prevent bad changes
- Common reasons a deployment can fail after review
- How to add deployment checks before merging

Hands-On Project

1. Create a feature branch and make a real code or config change
2. Open a pull request with a clear description
3. Add automated tests (e.g., unit test or build test)
4. Simulate a deployment failure (e.g., wrong config or missing variable)
5. Fix the issue, update the PR, and deploy successfully

Outcome

You will build a complete, production-ready pull request workflow with testing, review, and deployment validation.

Who Approved the Change That Took Down Payments?



Implement CODEOWNERS, protected branches, rulesets, and approval boundaries.

The Governance Flow

Code Change (Pull Request) → **CODEOWNERS** (Required Reviewers) → **Protected Branch** (Enforced Rules) → **Approval Boundaries** (Min. Reviews) → **Safe Deployment** (to Production)

Key Concepts

- **CODEOWNERS:** assign required reviewers by area
- **Protected Branches:** prevent direct or risky changes
- **Rulesets:** enforce rules (e.g., tests, approvals, status checks)
- **Approval Boundaries:** define who can approve and merge
- **Audit & Accountability:** trace who approved what and when

Hands-On Project

1. Create a repo for a payment service
2. Add a CODEOWNERS file (e.g., /payments/ → @payments-team)
3. Configure a protected branch (e.g., main)
4. Add rulesets (required reviews, tests, and status checks)
5. Test the workflow with a pull request and validate the approvals

Outcome

You will implement a secure, auditable GitHub workflow that controls who can change, approve, and deploy critical code.

Find the Exact Commit That Introduced the Incident



Use git bisect, test automation, and commit evidence to isolate a regression.

The Investigation Flow



Key Concepts

- What a regression is (and why it happens)
- How git bisect works (divide and find)
- Using automated tests with git bisect run
- Reading the commit evidence (diff, message, author, date)
- Confirming the root cause and creating a fix

Hands-On Project

1. Use a sample app with a known bug
2. Mark a known good commit (the last working version)
3. Mark a known bad commit (the broken version)
4. Run git bisect with an automated test script
5. Identify the exact commit, review the change, and fix it

Outcome

You will confidently trace a production incident to a single commit using git bisect, tests, and commit evidence.

Recover the Work Everyone Thought Was Deleted



Practice git reflog, detached HEAD recovery, reset, restore, and safe recovery methods.

The Recovery Flow



Key Concepts

- git reflog: view recent HEAD movements (your safety net).
- Detached HEAD: working on a commit without a branch.
- git reset: move branch to a specific commit (use with care).
- git restore: recover files or changes safely.
- Safe recovery: recover, verify, then commit to a new branch.

Hands-On Project

1. Delete or reset to "lose" some work (simulated).
2. Use git reflog to find the missing commit hash.
3. Recover the work (checkout, restore, or reset).
4. Create a new branch for the recovered work.
5. Commit, verify the files, and keep the history clean.

Outcome

You will confidently recover lost or deleted work, understand what happened, and use safe recovery methods in real projects.

Roll Back Production Without Rewriting Shared History



Compare revert, reset, hotfix branches, and emergency release procedures.

The Rollback Flow



Key Concepts

- **Revert:** create a new commit that safely undoes changes (keeps history).
- **Reset:** move the branch pointer (use only on local or private branches).
- **Hotfix Branch:** a short-lived branch for urgent production fixes.
- **Emergency Release:** a fast, controlled deployment to restore service.
- **Keep History Intact:** avoid rewriting shared (public) history.

Hands-On Project

1. Reproduce a production issue (simulated).
2. Use git revert to roll back the faulty change.
3. Create a hotfix branch, apply a targeted fix, and merge it.
4. Deploy the hotfix using an emergency release process.
5. Verify the fix in production and confirm stable operation.

Outcome

You will safely roll back a bad change, choose the right strategy (revert, reset, or hotfix), and run a real emergency release.

VERIQTA

The CI Pipeline That Trusted an Untrusted Pull Request



Examine GitHub Actions permissions, secrets, third-party actions, and workflow security.

The CI Security Flow

Pull Request (Untrusted (Contributor) → GitHub Actions (CI Pipeline) → Permissions & Secrets (Access Control) → Third-Party Actions (Trust & Risk) → Secure Pipeline (Hardened & Safe)

Key Concepts

- **Workflow Permissions:** control what the pipeline can do.
- **Secrets:** keep tokens and keys safe from untrusted PRs.
- **Third-Party Actions:** useful but can introduce supply-chain risk.
- **Untrusted Pull Requests:** code from external contributors is not fully trusted.
- **Pipeline Hardening:** apply least privilege, review actions, and restrict secret access.

Hands-On Project

1. Create a repo with a GitHub Actions workflow.
2. Simulate an untrusted pull request (from a fork).
3. Review and fix workflow permissions (least privilege).
4. Secure secrets (restrict access to trusted runs only).
5. Audit third-party actions and update to safe, pinned versions.

Outcome

You will build a secure GitHub Actions pipeline that runs safely, protects secrets, and reduces supply-chain risk.

The Release Tag That Did Not Match the Deployed Code



Build a traceable release using version tags, commit SHAs, artifacts, and deployment records.

The Traceability Flow

v1.4.0 → → → → →

- **Version Tag** (Release)
- **Commit SHA** (Exact Code State)
- **Artifacts** (Build Output (Immutable))
- **Deployment Record** (Who, When, What)
- **Verified Release** (Tag = Code = Deploy)

Key Concepts

- **Version Tag:** a named, immutable snapshot of a specific commit.
- **Commit SHA:** the unique ID of the exact code (e.g., a1b2c3d).
- **Artifacts:** the built, versioned output (image, binary, package).
- **Deployment Record:** evidence of what was deployed, when, and by whom.
- **Release Traceability:** link tag → commit → artifact → deployment for full auditability.

Hands-On Project

1. Create a release tag (e.g., v1.4.0) for a known commit.
2. Build and store a versioned artifact (e.g., Docker image).
3. Deploy to a test environment and record the deployment details.
4. Verify the deployed code matches the tag and commit SHA.
5. Generate a simple release report (tag, SHA, artifact, and deployment).

Outcome

You will create a fully traceable release where the tag, commit, artifact, and deployed code always match.

One Repository, Six Teams, Constant Merge Conflicts



Design repository ownership, monorepo boundaries, review paths, and change coordination.

The Collaboration Flow

Six Teams (Multiple Changes) → Repository (Monorepo) → Ownership (CODEOWNERS) → Review Paths (Team Rules) → Merged (Safe & Coordinated)

Key Concepts

- **Repository Ownership:** assign clear owners per area (e.g., /api, /web, /infra).
- **Monorepo Boundaries:** define module boundaries to reduce unintended changes.
- **Review Paths:** route changes to the right teams automatically.
- **Change Coordination:** plan, communicate, and sequence related changes.
- **Conflict Reduction:** use smaller, focused pull requests and keep branches short-lived.

Hands-On Project

1. Create a monorepo with six team folders (e.g., /api, /web, /infra, /data, /platform, /docs).
2. Add a CODEOWNERS file to define repository ownership.
3. Set up branch protection and review requirements per area.
4. Simulate conflicting changes across two teams and resolve them.
5. Create a simple change coordination plan (PR order, reviewers, and merge strategy).

Outcome

You will run a multi-team repository with clear ownership, fewer merge conflicts, and a predictable review and merge process.

VERIQTA

The Merge Queue That Prevented a Broken Main Branch



Simulate competing pull requests and use merge queue checks to detect incompatible changes.

The Merge Queue Flow



Key Concepts

- Merge Queue: a controlled queue that tests PRs before merging to main.
- Competing PRs: multiple PRs that can affect the same code (e.g., shared files).
- Queue Checks: automated build, test, and compatibility checks per PR.
- Incompatible Changes: PRs that can't be merged together safely.
- Stable Main: only PRs that pass all checks are merged.

Hands-On Project

1. Create two competing PRs (e.g., PR #101 and PR #102) that change the same code.
2. Enable a merge queue on the main branch (GitHub).
3. Configure required checks (build and tests).
4. Observe the queue: one PR merges, the other is held due to a conflict or failing checks.
5. Fix the incompatible PR and watch it merge successfully.

Outcome

You will use a merge queue to manage competing pull requests, catch incompatible changes early, and keep the main branch always stable.

VERIQTA

The Deployment That Worked Only on One Engineer's Laptop



Investigate tracked files, .gitignore, line endings, dependencies, and reproducibility.

The Investigation Flow



1. **Works on One Laptop** (Inconsistent)
2. **Investigate Differences** (Files, Env, Config)
3. **Check Tracking & .gitignore** (Include/Exclude)
4. **Verify Dependencies** (Versions & Lockfile)
5. **Reproduce Everywhere** (Consistent Build)

Key Concepts

- **Tracked Files:** only commit required and deterministic files (e.g., code, config, lockfiles).
- **.gitignore:** exclude local, build, cache, and OS-specific files (e.g., node_modules/, .env, dist/).
- **Line Endings:** keep consistent (LF) across the team (.gitattributes).
- **Dependencies:** pin versions (e.g., requirements.txt, package-lock.json).
- **Reproducibility:** same code, same dependencies, same result on any machine.

Hands-On Project

1. Clone a simple app that works on one laptop but fails for others.
2. Inspect tracked files and update .gitignore to remove local artifacts.
3. Fix line endings (add a .gitattributes file).
4. Pin and install dependencies (e.g., npm ci or pip install -r requirements.txt).
5. Rebuild and run the app on a clean environment (e.g., Docker or a fresh clone).

Outcome

You will produce a clean, reproducible project that builds and deploys consistently on any machine.

The Open Source Contribution That Failed Maintainer Review



Work through issues, forks, upstream synchronization, pull request etiquette, and review corrections.

The Contribution Flow

Issues (Find & Discuss) → **Fork** (Create Your Copy) → **Sync Upstream** (Keep in Sync) → **Pull Request** (Propose Changes) → **Review & Iterate** (Address Feedback, Get Merged)

Key Concepts

- **Issues:** start by discussing the change with maintainers.
- **Forks:** work in your own copy of the repository.
- **Upstream Sync:** keep your fork up to date with the main repo.
- **Pull Request (PR):** a clear, small, and well-described change.
- **Review Corrections:** respond to feedback, make updates, and resubmit.

Hands-On Project

1. Find a "good first issue" in a real open source repo (e.g., Kubernetes, Prometheus, or a tool).
2. Fork the repo and create a feature branch.
3. Make a small, meaningful fix (e.g., docs, bug, or test).
4. Open a pull request with a clear description and follow the repo's rules.
5. Address maintainer feedback, update your PR, and get it merged.

Outcome

You will successfully contribute to an open source project by following the right process, communication, and review etiquette.

The Emergency Patch That Never Reached the Main Branch



Design a hotfix process that reconciles production, release, and development branches.

The Hotfix Flow



Key Concepts

- **Branching Model:** main (stable), release (next), develop (ongoing), hotfix/* (urgent fixes).
- **Hotfix:** a small, high-priority fix for a production issue.
- **Reconciliation:** merge the hotfix into both release and develop (or main).
- **Same Code Everywhere:** ensure the fix is not lost in other branches.
- **Verification:** test the patch before and after merging.

Hands-On Project

1. Start with three branches: main, release/1.4, and develop.
2. Create a hotfix branch: `git checkout -b hotfix/1.4.1 release/1.4`.
3. Apply a simple fix and commit it.
4. Merge the hotfix to release:
   `git checkout release/1.4 && git merge hotfix/1.4.1`
5. Merge the same fix to main and develop:
   `git checkout main && git merge hotfix/1.4.1`
   `git checkout develop && git merge hotfix/1.4.1`

Outcome

You will have a tested, deployed hotfix in production and the same fix safely integrated into both release and development branches.

The GitHub Organization With Too Much Access



Audit repository permissions, automation credentials, branch protections, and ownership.

The Access Audit Flow

Audit Access (Users, Teams, Roles) → **Review Permissions** (Least Privilege) → **Check Automation** (Tokens & Secrets) → **Enforce Branch Protections** (Checks, Reviews, Rules) → **Confirm Ownership** (Teams & Maintainers)

Key Concepts

- **Repository Permissions:** control who can read, write, and admin.
- **Automation Credentials:** tokens (e.g., PAT, GitHub App), secrets in Actions, and their scope.
- **Branch Protections:** rules like required reviews, status checks, and restricted pushes.
- **Ownership:** clear maintainers, code owners, and team responsibilities.
- **Least Privilege:** give only the minimum access required.

Hands-On Project

1. Audit your GitHub organization (or a test org) members and team access.
2. Review automation credentials (e.g., GitHub Actions secrets, PATs) and reduce their scope.
3. Enable branch protection on main (e.g., required reviews and passing checks).
4. Set CODEOWNERS for key folders or components.
5. Document ownership and access (GitHub Teams, maintainers, and roles).

Outcome

You will run a secure, well-governed GitHub organization with the right access, strong protections, and clear ownership.

Build a Production-Grade GitHub Delivery System



Integrate CI, release approvals, security checks, deployment evidence, and rollback procedures.

The Delivery Pipeline Flow

Code & PR (GitHub) → CI (Build, Test, Scan) → Security Checks (SAST, SCA, Secrets) → Release Approval (Manual or Policy) → Deploy (Staging → Prod) → Rollback (If Needed)

Key Concepts

- **CI/CD Integration:** automated build, test, and deploy with GitHub Actions.
- **Release Approvals:** human or policy-based gates before prod.
- **Security Checks:** code, dependency, and secret scanning (SAST, SCA).
- **Deployment Evidence:** record what was deployed, when, and by whom (commit, tag, artifact).
- **Rollback Procedures:** fast, tested process to return to a stable version.

Hands-On Project

1. Create a GitHub Actions pipeline (build, test, and scan).
2. Add security checks (e.g., CodeQL and dependency scan).
3. Set a release approval before deploying to production.
4. Deploy with a version tag and store deployment evidence (artifact, logs, commit SHA).
5. Simulate an issue and perform a rollback to the previous stable release.

Outcome

You will have a secure, end-to-end GitHub delivery system for real production use, with approvals, traceability, and a reliable rollback.

Final Project: Recover a Compromised Repository and Failed Release



Complete a realistic incident involving leaked credentials, conflicting changes, a broken release, and recovery evidence.

The Incident & Recovery Flow

Detect Breach (Leaked Secret) → **Resolve Conflicts** (Merge & Test) → **Fix Broken Release** (Build & Validate) → **Restore & Recover** (Rotate, Rebuild) → **Provide Evidence** (Logs, PRs, Release)

Key Concepts

- **Incident Response:** detect, contain, fix, and recover from a repo breach.
- **Credential Hygiene:** rotate, revoke, and replace exposed secrets.
- **Merge Conflict Resolution:** find conflicts, reconcile changes, and stabilize code.
- **Release Recovery:** rebuild, retest, and redeploy a working release.
- **Recovery Evidence:** document actions with logs, PRs, and release notes.

Hands-On Project

1. Investigate the incident (find the leaked credential and its impact).
2. Revoke and rotate the leaked secret (e.g., GitHub, cloud, or API key).
3. Resolve the merge conflict and restore a clean, buildable main.
4. Fix the failed release and redeploy a working version.
5. Document the recovery with evidence (incident summary, actions, PRs, logs, and final release).

Outcome

You will recover a real-world GitHub incident, restore a broken release, and produce clear, verifiable recovery evidence.